Introduction To JUnit

By Viren

1. Unit Testing
2. Test Phases
3. Test Fixture
4. Junit Introduction
5. Junit Features
6. Annotations
7. Assert Statements
8. Few Best Practice

- **Unit testing** is a software development process in which the smallest testable parts of an application, called units, are individually and independently scrutinized for proper operation.
- Unit testing increases confidence in changing/maintaining code.
- Every time any code is changed, the likelihood of any defects due to the change being promptly caught is very high.
- A ***unit test*** is a piece of code written by a developer to test a unit. e.g. a method or a class, (local tests).
- The percentage of code which is tested by unit tests is typically called ***test coverage***.

JUnit

“first testing then coding”

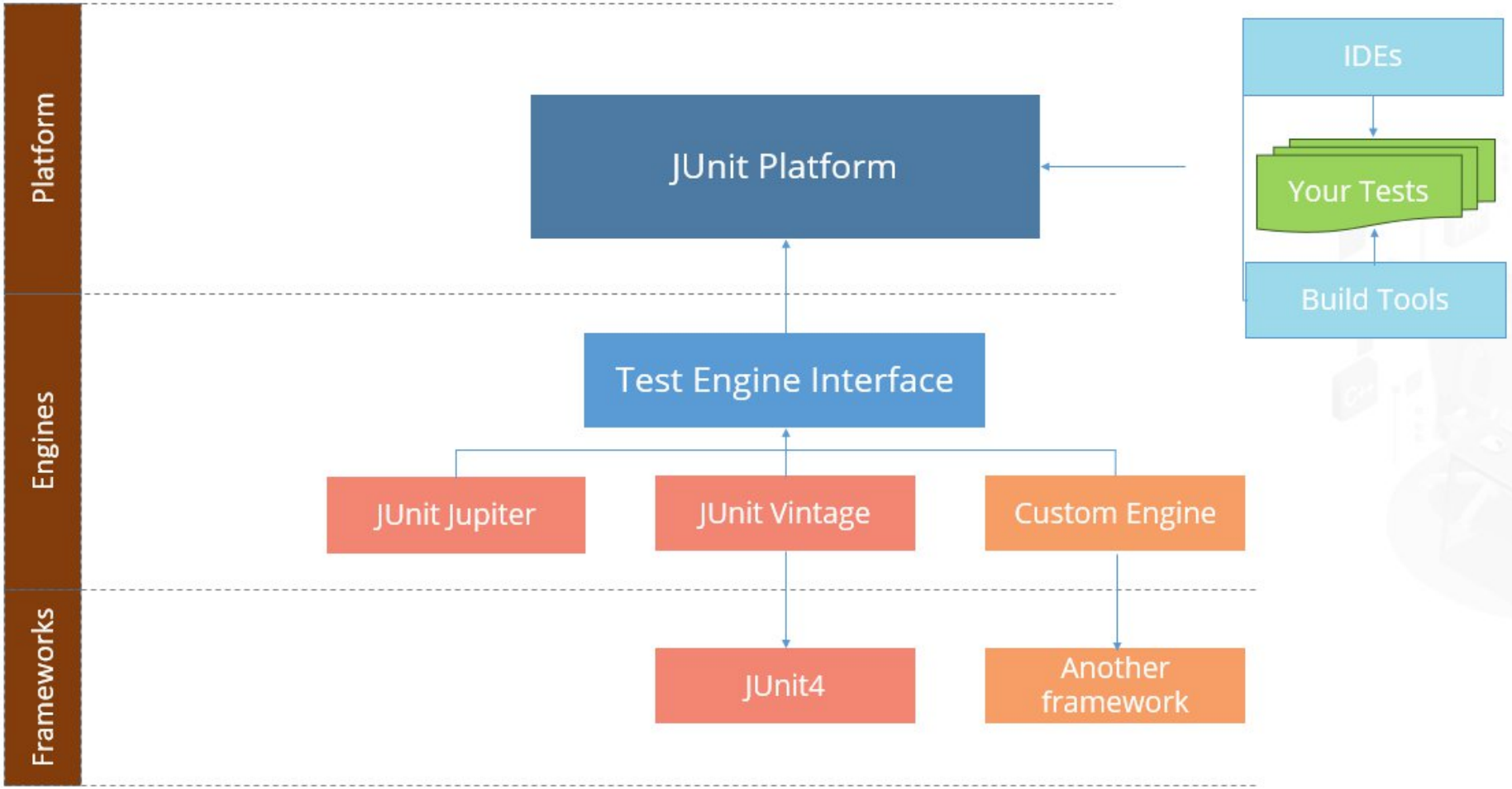
- Junit is a **unit testing** framework for the Java Programming Language.
- JUnit is the Java version of the **xUnit** architecture for unit- and regression-testing frameworks.
- Junit is a standard tool for **test driven development(TDD)** in Java.
- Junit comes pre-bundled with several Java IDEs (**Eclipse**, BlueJ, Jbuilder, etc).
- JUnit uses Java's **reflection** capabilities (Java programs can examine their own code)
- Beck and Gamma (of design patterns Gang of Four) developed JUnit on a flight from Zurich to Washington, D.C.

features

“yes, Its possible!”

- Provides **Fixtures** to set fixed state of the software under test to be used as a baseline for running tests
- Provides **Annotation** to identify the test methods.
- Provides **Assertions** for testing expected results.
- Provides **TestRunner** for running tests(main()).
- Provides **TestSuite** for organizing and running groups of tests
- Junit shows test progress in a bar that is green if test is going fine and it turns red when a test fails.

JUnit 5 Architecture

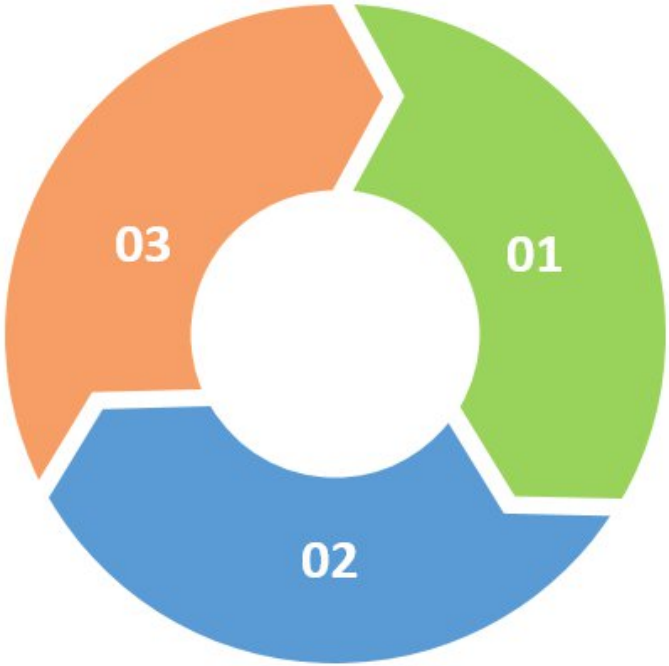


JUnit 5 Architecture

JUnit 5 is split into three different subprojects.

JUnit Vintage

Allows running of JUnit 4 tests with JUnit 5



JUnit Platform

Provides build plugins and an API for writing test engines

JUnit Jupiter

Provides new API for writing tests and extensions in JUnit 5

@Annotations

"tell JUnit how to deal with the method at hand"

Annotation(JUnit4)	Description
@Test public void method()	The @Test annotation identifies a method as a test method.
@Test (expected = Exception.class)	Fails, if the method does not throw the named exception.
@Test(timeout=100)	Fails, if the method takes longer than 100 milliseconds.
@Before public void method()	This method is executed before each test. It is used to can prepare the test environment (e.g. read input data, initialize the class).
@After public void method()	This method is executed after each test. It is used to cleanup the test environment (e.g. delete temporary data, restore defaults). It can also save memory by cleaning up expensive memory structures.
@BeforeClass public static void method()	This method is executed once, before the start of all tests. It is used to perform time intensive activities, for example to connect to a database. Methods annotated with this annotation need to be defined as static to work with JUnit.
@AfterClass public static void method()	This method is executed once, after all tests have been finished. It is used to perform clean-up activities, for example to disconnect from a database. Methods annotated with this annotation need to be defined as static to work with JUnit.
@Ignore	Ignores the test method. This is useful when the underlying code has been changed and the test case has not yet been adapted. Or if the execution time of this test is too long to be included.

Moving from JUnit 4 to JUnit 5

JUnit 5 provides a gentle migration path via the JUnit Vintage test engine, which allows for execution of legacy test cases on top of the JUnit platform.

Feature	JUnit 4	JUnit 5
Annotation package	org.JUnit	org.JUnit.Jupiter.api
Declaring a test	@Test	@Test
Setup for all tests	@BeforeClass	@BeforeAll
Setup per test	@Before	@BeforeEach
Tear down for all tests	@AfterClass	@AfterAll
Tear down per test	@After	@AfterEach
Tagging and filtering	@Category	@Tag
Disabling a test method or a class	@Ignore	@Disabled
Nested tests	NA	@Nested
Repeated tests	Using custom rule	@Repeated
Dynamic tests	NA	@TestFactory
Test templates	NA	@TestTemplate

Annotations

The following annotations are used for configuring tests in JUnit5:

Annotation	Description
@Test	Denotes a test method
@DisplayName	Declares a custom display name for the test class or test method
@BeforeEach	Denotes that the annotated method should be executed before each test method
@AfterEach	Denotes that the annotated method should be executed after each test method
@BeforeAll	Denotes that the annotated method should be executed before all test methods
@AfterAll	Denotes that the annotated method should be executed after all test methods
@Disable	Used to disable a test class or test method
@Nested	Denotes that the annotated class is a nested and non-static test class
@Tag	Declares tags for filtering tests
@ExtendWith	Registers custom extensions

Assert Statements

"compare the expected result with the actual result"

- **JUnit** provides static methods in the Assert class to test for certain conditions.
- These ***assertion methods*** typically start with assert and allow you to specify the error message, the expected and the actual result.
- Example:

```
@Test
public void testMultiply() {
MyClass tester = new MyClass();
assertEquals("10 x 5 must be 50", 50, tester.multiply(10, 5));
}
```

- An ***assertion method*** compares the actual value returned by a test to the expected value, and throws an AssertionException if the comparison test fails.

Statement Parameters in [] brackets are optional.	Description
fail(String)	**Lets the method fail.** Might be used to check that a certain part of the code is not reached. Or to have a failing test before the test code is implemented. The String parameter is optional.
assertTrue([message], boolean condition)	Checks that the Boolean **condition is true**.
assertFalse([message], boolean condition)	Checks that the Boolean **condition is false.**
assertEquals([String message], expected, actual)	Tests that **two values are the same**. Note: for arrays the reference is checked not the content of the arrays.
assertEquals([String message], expected, actual, tolerance)	Test that **float or double values match**. The tolerance is the number of decimals which must be the same.
assertNull([message], object)	Checks that the **object is null**.
assertNotNull([message], object)	Checks that the **object is not null**.
assertSame([String], expected, actual)	Checks that **both variables refer to the same object**.
assertNotSame([String], expected, actual)	Checks that **both variables refer to different objects.**

Note:

You should provide meaningful messages in assertions so that it is easier for the developer to identify the problem. This helps in fixing the issue, especially if someone looks at the problem, which did not write the code under test or the test code.

Happy Testing!